10 / 20 — Phase 10 Assessment: Production Engineering

Covers: lessons 01 through 19 of Phase 10: repository organization, deployment workflows, environment configuration, observability and logging, error monitoring, security (XSS, CSRF, CSP), dependency security, accessibility as architecture, internationalization, resilience and degradation, design system governance, backward compatibility, application migrations, multi-team collaboration, technical debt, architecture decision records, documentation, continuous delivery, and incident debugging.

Level: Professional. **Recommended time:** 3.5 hours (2.5 h closed-notes for sections A–D, then 1 h for section E; or take it open-notes if you want a learning exercise rather than a gate). **Total:** 100 points.

> This assessment need not follow the 15-section lesson template. Attempt every question before reading the answer key in Part 2.

Part 0 — How to use this assessment

1. Work without looking at the lessons first. Write answers in full sentences or code; do not just list keywords.
2. Mentally execute code; do not run it until you have committed to a prediction.
3. Score yourself with the rubric in Part 3 *after* reading the answer key. Be strict: partial credit is defined per question.
4. Record category totals. The promotion criteria (Part 4) require **≥ 80% overall AND ≥ 70% in every category**.

Category map

Category	Sections	Points	What it measures
Explanation	A (A1–A10)	25	Can you explain production-engineering concepts precisely, with causal reasoning?
Implementation	C (C1–C4)	25	Can you write correct, production-quality code and documents?
Debugging	B (B1–B5) + D (D1–D3)	25	Can you predict behavior, read evidence, and find root causes?
Reasoning	E (E1–E4)	25	Can you make and defend architectural and organizational decisions?

Part 1 — Questions

Section A — Conceptual questions (Explanation, 25 points; 2.5 each)

A1 (Lessons 01, 14). A company has eight React teams and is deciding between one monorepo and eight repositories. State the main trade-offs, and explain what must be true *in either choice* for team boundaries to be real rather than aspirational.

A2 (Lessons 02, 03). Explain "build once, deploy many" for a React SPA, why it conflicts with the common practice of baking `VITE_*` / `REACT_APP_*` variables into the bundle, and what the consequence is for secrets in any frontend configuration.

A3 (Lesson 04). Distinguish logs, metrics, and traces in the context of a React frontend plus a BFF. Explain what makes correlation across them possible, and name two privacy/cost hazards of client-side telemetry.

A4 (Lesson 05, 19). Why do production error reports need a release identifier and uploaded source maps? Which errors do React error boundaries *not* catch, and how would you make sure those are still reported?

A5 (Lesson 06). Explain how XSS and CSRF differ in mechanism and in which defenses address each. Explain why React's default escaping does not make an app XSS-proof, and what role CSP plays.

A6 (Lesson 07). A teammate says "we're safe because `npm audit` is clean." Explain what that statement does and does not establish, and list four practices that reduce dependency risk beyond running an audit.

A7 (Lesson 08). What does it mean to treat accessibility as *architecture* rather than as a testing phase? Give three concrete architectural mechanisms and one thing automated accessibility tooling cannot verify.

A8 (Lesson 09). List four reasons string concatenation and hard-coded formatting break internationalization, and name the platform and library mechanisms that address each.

A9 (Lesson 10). Explain why naive retries can make an outage worse, and describe a retry policy that is safe. Include which requests may be retried, what delay strategy you use, and how you bound the total work.

A10 (Lessons 16, 17). Explain (a) why accepted ADRs are not edited and how a changed decision is recorded, and (b) the four Diátaxis documentation types and which of them a runbook and an ADR are.

Section B — Code prediction (Debugging, 10 points; 2 each)

B1 (Lesson 18). Feature-flag bucketing is deterministic: `bucket(userId, flagKey)` returns an integer in `[0, 100)`, and a user is enabled when `bucket < percent` or the user is in the allow list. For flag `x`, users have these buckets: `ana = 12`, `ben = 37`, `cho = 64`, `dev = 91`. The rule changes over time:

1. `percent: 30, allow: []`
2. `percent: 65, allow: []`
3. `percent: 0, allow: ['dev']`
4. `percent: 100, allow: []`

For each stage, list who is enabled. Then answer: (a) Does any stage turn *off* a user who was on in the previous *percentage* stage (stages 1→2, 2→4)? (b) For a *different* flag `y` with a different salt, could `ana` have a bucket of 88, and why is salting by flag key desirable?

B2 (Lesson 12). Given:

```ts
import { z } from 'zod';

const V1 = z.object({ v: z.literal(1), compact: z.boolean() });
const V2 = z.object({
  v: z.literal(2),
  density: z.enum(['comfortable', 'compact']),
  theme: z.enum(['light', 'dark', 'system']).default('system'),
});
type Prefs = z.infer<typeof V2>;
const DEFAULTS: Prefs = { v: 2, density: 'comfortable', theme: 'system' };

function load(text: string | null): Prefs {
  try {
    if (text === null) return DEFAULTS;
    const raw: unknown = JSON.parse(text);
    const a = V1.safeParse(raw);
    if (a.success) return { v: 2, density: a.data.compact ? 'compact' : 'comfortable', theme:
'system' };
    const b = V2.safeParse(raw);
    return b.success ? b.data : DEFAULTS;
  } catch {
    return DEFAULTS;
  }
}
```

What does `load` return for each input?

1. `null`
2. `'{"v":1,"compact":true}'`
3. `'{"v":2,"density":"compact"}'`
4. `'{"v":3,"density":"compact","theme":"dark"}'`
5. `'{"v":2,"density":"tiny"}'`
6. `'{oops'`

Then state what a user experiences if they use a *newer* app version that writes `v: 3`, and then the team rolls back to this code.

B3 (Lessons 02, 18). Response headers: `/index.html` → `Cache-Control: no-cache`, `ETag` present. `/assets/app.AAA111.js` and all hashed assets → `Cache-Control: public, max-age=31536000, immutable`. A user loads the site at 09:00 (app.AAA111.js, plus lazy chunk `dash.AAA222.js` not yet requested). At 10:00 a deploy ships `app.BBB111.js` / `dash.BBB222.js`, and the deploy script **deletes** the AAA assets. At 10:30 the user (who never reloaded) clicks a link that triggers the lazy import of the dashboard.

(a) Which request(s) go to the network on a *normal reload* at 10:45 and what are the responses? (b) What happens at 10:30 for the user who did not reload, assuming the server's SPA fallback returns `index.html` with status 200 for any unknown path? (c) Name two changes that prevent the failure in (b).

B4 (Lesson 19). Predict what the user sees and where each error surfaces. All components are wrapped in a single `<Boundary>` around `<Bomb />` **only** (no other boundary anywhere); `Boundary` is a standard class error boundary with a fallback that renders `<p role="alert">Something broke</p>`.

```
function Bomb({ mode }: { mode: 'render' | 'click' | 'timer' | 'promise' }) {
  const [n, setN] = useState(0);
  if (mode === 'render' && n === 0) throw new Error('render');
  return (
    <button
      onClick={() => {
        if (mode === 'click') throw new Error('click');
        if (mode === 'timer') setTimeout(() => { throw new Error('timer'); }, 0);
        if (mode === 'promise') Promise.reject(new Error('promise'));
        setN(1);
      }}
    >
      go {n}
    </button>
  );
}
```

For each of the four modes (rendered as `<Boundary><Bomb mode={mode} /></Boundary>`): what is on screen initially, what happens after one click, and which global handler (if any) sees the error? Then: what changes for `mode="render"` if the `Boundary` is removed?

B5 (Lesson 12). A library changes the following between v1 and v2. For each, say whether it is breaking for TypeScript consumers who compile against the library (assume the consumers' existing code is shown), and why.

1. `interface ListProps { items: Item[] }` → `interface ListProps { items: readonly Item[] }`, where consumers *pass* arrays.
2. `interface ListProps { onSelect: (id: string) => void }` → `interface ListProps { onSelect: (id: string, item: Item) => void }`, where consumers *pass* handlers written as `(id) => ...`.
3. `function getTone(): 'info' | 'warn'` → `function getTone(): 'info' | 'warn' | 'error'`, where consumers use an exhaustive `switch` with `assertNever` in `default`.
4. `interface ListProps { dense?: boolean }` → `interface ListProps { dense: boolean }`.

Section C — Implementation (Implementation, 25 points)

C1 (Lessons 11, 12; 8 points). `@acme/ds` `Button` currently accepts `variant: 'primary' | 'secondary' | 'link'`. The team wants `tone: 'primary' | 'secondary' | 'ghost'`. Write the **expand-phase** `Button` (React 19 function component with `ref` as a prop) such that: both props work; `tone` wins when both are given; `'link'` maps to `'ghost'`; `variant` is typed as deprecated with JSDoc; a development-only warning is emitted at most once per page load; there is no warning in production. State precisely what version bump this release is and what the contract (removal) release requires.

C2 (Lesson 06; 6 points). Write a `Content-Security-Policy` header for a React SPA with these facts: all scripts are first-party bundles served from the same origin, except an analytics script at `https://analytics.example/a.js`; XHR/fetch goes to the same origin and `https://api.example.com`; images come from the same origin, `data:` URIs, and `https://img.example`; there are no inline `<script>` or `<style>` elements and no inline event handlers; the page must never be framed; there are no plugins. Explain each directive you use, and describe how to roll it out safely. State what you would change if a runtime CSS-in-JS library that injects `<style>` elements were introduced.

C3 (Lesson 10; 6 points). Write a TypeScript `fetchJson<T>(url, opts)` helper with: a per-attempt timeout; the caller's `AbortSignal` honored; retries (maximum 3 attempts total) only for idempotent methods and for retryable failures (network errors, 408, 429, 502, 503, 504); exponential backoff with full jitter; `Retry-After` support in seconds; no retries after the caller aborts; and a thrown error class carrying `status` when the response is not OK. Do not use `AbortSignal.timeout` or `AbortSignal.any` (assume you must support browsers that lack them).

C4 (Lesson 16; 5 points). Write a complete, concise ADR (Nygard-style with drivers and options) for the decision: "*Store the session in an httpOnly, Secure, SameSite cookie managed by a backend-for-frontend, instead of keeping access tokens in* `localStorage` *in the SPA.*" Invent plausible context facts with numbers. Include at least three options, honest negative consequences, enforcement, and a revisit trigger.

Section D — Debugging challenges (Debugging, 15 points; 5 each)

D1 (Lessons 02, 18). After a deploy at 13:00, support reports that many users still see the old UI the next morning; a hard refresh fixes it. Evidence:

```
$ curl -sI https://app.example.com/ | grep -iE 'cache-control|age|etag|x-cache'
cache-control: public, max-age=86400
age: 41210
etag: "7c1f"
x-cache: HIT

$ curl -sI https://app.example.com/assets/app.9f3a1c.js | grep -iE 'cache-control|age'
cache-control: public, max-age=300
age: 120

$ curl -s https://app.example.com/version.json
{"build":"a1b2c3"}      # (the new build is a1b2c3; the previous build was 77aa00)
```

Diagnose all problems visible in this output, explain precisely why users see the old UI, give the fixes (including how to clear the immediate impact), and describe how you'd verify.

D2 (Lesson 06; security review). You review this PR in a banking dashboard. List every security problem you can find, classify each (XSS, CSRF, token exposure, etc.), explain the exploit, and give a fix for each.

```
function Note({ html }: { html: string }) {
  return <div dangerouslySetInnerHTML={{ __html: html }} />;       // html comes from
/api/notes (user-authored)
}

function Contact({ website }: { website: string }) {               // `website` is user-
supplied profile data
  return <a href={website} target="_blank">Visit website</a>;
}

export async function login(user: string, pass: string) {
  const res = await fetch('/api/login', { method: 'POST', body: JSON.stringify({ user, pass })
});
  const { token } = await res.json();
  localStorage.setItem('token', token);
}

export async function transfer(to: string, amount: number) {
  return fetch('/api/transfer', {                                  // server also accepts
cookie sessions
    method: 'POST',
    credentials: 'include',
    body: JSON.stringify({ to, amount }),                          // no content-type header
set
  });
}
```

D3 (Lessons 09, 19; incident). At 09:10 UTC release 3.8.0 ships a "delivery date" step in checkout behind the flag `checkout.delivery-date` at 20%. At 09:31 an alert fires: error sessions on `/checkout` at 2.8% (baseline 0.3%). Your dashboard breakdowns:

Dimension	Value	Error sessions (30 min)
Release	3.8.0	1,904
Release	3.7.2	14
Flag `checkout.delivery-date`	on	1,897
Flag `checkout.delivery-date`	off	21
Browser	Chrome 12x	900
Browser	Safari 17	640
Browser	Firefox	364
Locale	`en-US`	0
Locale	`en-GB`	2
Locale	`de-DE`	1,203

Dimension	Value	Error sessions (30 min)
Locale	`tr-TR`	689
Locale	other	10

The top symbolicated error is `RangeError: Invalid time value` at `toISOString` in `deliveryDate.ts:28`. The relevant code:

```
// deliveryDate.ts (excerpt)
export function toApiDate(input: string): string {        // `input` is the date picker's
*display* value
  return new Date(input).toISOString().slice(0, 10);      // line 28
}
// The picker shows dates formatted with Intl.DateTimeFormat(locale), e.g. de-DE "15.03.2026",
en-US "3/15/2026"
```

(a) Which breakdown(s) are discriminating, and which are noise? (b) What is the root cause and why does `en-US` not fail? (c) What is the best immediate mitigation and why? (d) Give the permanent fix and two guardrails (one for tests, one for detection). (e) Which dimension was missing from your error context that would have made this faster, if any?

Section E — Architecture and reasoning (Reasoning, 25 points)

E1 (Lessons 11–13; 7 points). Your design system `@acme/ds` is used by 30 applications (six teams). You must (1) rename the token `color.text.default` → `color.text.primary`, and (2) replace `Button`'s `variant` with `tone` (E1/C1), and (3) raise the minimum React peer from 18.3 to 19. Design the rollout over one quarter: versioning (which are minor/major), expand/contract sequencing, tooling, communication, measurement of progress, the handling of laggard teams, and rollback. State what you would **not** do.

E2 (Lessons 14–16; 6 points). A 60-engineer organization has six stream-aligned teams in one monorepo. Symptoms: a `shared/utils` package with 4,000 lines and no owner; merge conflicts daily in shared files; main is red about six hours a week; engineers say "nobody knows why we use X". Propose an ownership model, boundary enforcement, merge/review mechanics, debt handling, and decision-recording practice. Specify measurable targets and how you'd sequence the changes so teams can still deliver.

E3 (Lessons 04, 05, 18, 19; 6 points). You must replace the authentication provider for a customer-facing app (about 2 million monthly users). Design the delivery and verification plan: what is flagged and how, rollout stages and cohort choices, what you instrument beforehand (specific metrics and dimensions), abort criteria, rollback strategy including stale clients and sessions, communication, and what you rehearse in advance.

E4 (Lessons 07–10, 12; 6 points). The company will launch in two new markets: Germany (German UI, EUR, strict privacy expectations) and the UAE (Arabic UI, right-to-left layout, AED). Identify the architectural work required across internationalization, accessibility, security/privacy (client-side storage and telemetry), resilience, and design-system governance. Prioritize and explain what you would verify before launch and how you'd keep the cost of future markets low.

Part 2 — Answer key

> Attempt the assessment first. This key gives model answers and the evidence expected for full credit.

Section A

A1. *Model answer.* Monorepo trade-offs: atomic cross-cutting changes (one PR across packages), shared tooling and dependency versions, easier refactoring and discoverability; costs: CI scaling (needs affected-only builds, caching, merge queue), repo-wide permissions/ownership complexity, risk of accidental coupling. Eight repos: autonomy in tooling and release cadence, clear permission boundaries, smaller CI; costs: version skew of shared libraries, cross-repo changes require coordinated releases, duplicated config, harder discovery. *In either choice:* explicit ownership (CODEOWNERS/service catalog), public APIs per package (package `exports`, entry points), automated boundary enforcement (lint/dependency analysis), typed contracts, and an agreed review/release process. Boundaries must be enforced by tooling or they erode (the monorepo does not remove coupling; multi-repo does not guarantee autonomy if repos import each other's internals). *Must-haves:* (1) accurate trade-offs for both options, (2) ownership + explicit public interfaces, (3) enforcement via automation (not just convention). *Common wrong answer:* "monorepo means one team owns everything" or "multi-repo means decoupled".

A2. *Model answer.* Build the artifact once, test that exact artifact, promote the same bytes through environments, so test evidence applies to production. Bundlers replace `import.meta.env.VITE_X` / `process.env.REACT_APP_X` with literal values at build time, so changing the environment means rebuilding, and a "staging build" differs from a "production build". Instead use runtime configuration (a config JSON fetched at start, values injected into `index.html` at deploy time, or server-provided bootstrap) so one artifact serves all environments. Consequence for secrets: anything delivered to the browser, whether baked into the bundle or fetched as runtime config, is readable by users. Secrets (API keys with write power, private tokens) must stay on a server (BFF/backend); the frontend may hold only public identifiers and endpoints; variable-prefix conventions like `VITE_` exist to mark what *will be exposed*. *Must-haves:* build-once rationale, build-time substitution mechanism, no secrets in client config.

A3. *Model answer.* Logs: discrete events with context (client logs are often limited/sampled; the BFF has structured logs); metrics: aggregated numbers over time (error rate, latency percentiles, Web Vitals via RUM, conversion funnel), cheap to alert on; traces: a causal tree of spans for one request/interaction across services. Correlation uses a shared identifier: W3C Trace Context `traceparent` propagated from browser `fetch` through BFF to services, plus a session/correlation ID in logs and error reports and a release identifier. Hazards: PII leakage (URLs with IDs/emails, form values, session replay of sensitive inputs), consent requirements, and cost/cardinality (high-cardinality labels in metrics, unsampled verbose logs, telemetry volume from loops; also performance overhead on the main thread/network). *Must-haves:* distinction with use cases; correlation mechanism; two valid hazards.

A4. *Model answer.* Production code is minified; the stack in an error report is meaningless without source maps matched to the *exact build*, so the SDK tags events with a release ID and the pipeline uploads maps (privately) per release; a release ID also lets you answer "did this deploy cause it?" and group/regress by version. Error boundaries do not catch errors in event handlers, asynchronous code (timeouts, promises), server rendering, or in the boundary itself. Cover them via `window.addEventListener('error')`, `'unhandledrejection'`, the data layer's error states, and (React 19) root options such as

`onUncaughtError` / `onCaughtError` / `onRecoverableError` ⚠ verify names; choose one place to report to avoid duplicates. *Must-haves:* release + source maps rationale; the list of uncaught categories; a reporting mechanism for them.

A5. *Model answer.* XSS: attacker-controlled script executes in your origin (injected via unsanitized HTML, unsafe URLs, vulnerable dependencies/third-party scripts); it can read DOM/storage, act as the user, and bypass CSRF defenses because it is same-origin. CSRF: a malicious *other* site causes the victim's browser to send an authenticated request to your site, exploiting ambient credentials (cookies); defenses: `SameSite` cookies, anti-CSRF tokens or custom headers verified server-side, checking `Origin`, requiring non-simple content types. React escapes text interpolation, but `dangerouslySetInnerHTML`, user-controlled URLs (`javascript:` in `href`), `eval`-like patterns, unsafe third-party widgets, server-rendered HTML, and vulnerable packages bypass that. CSP is defense-in-depth: restricts where scripts load/execute, reduces impact of an injection (blocks inline scripts/untrusted origins), and `frame-ancestors` handles clickjacking; it does not replace sanitization or output encoding and is hard to deploy correctly. *Must-haves:* two mechanisms distinguished; at least two React bypasses; CSP as layered defense.

A6. *Model answer.* Clean `npm audit` means no *known, published* advisories for the resolved versions in the registry data at that time (and it only covers dependencies the tool can resolve). It does not cover zero-days, malicious packages (typosquatting, compromised maintainers), install scripts, unreachable-vs-reachable vulnerabilities, vulnerable transitive code not yet reported, license issues, or abandoned packages. Practices: committed lockfile with `--frozen-lockfile` / `npm ci`; automated update PRs (Renovate/Dependabot) with CI; review of dependency diffs for sensitive packages; minimizing dependencies; pinning CI actions by SHA and least-privilege CI tokens; disabling or reviewing install scripts; provenance/signature checks and SBOM generation where available; scoped registries/private mirrors; periodic removal of unused deps; monitoring advisories with reachability triage. Any four. *Must-haves:* what audit covers/doesn't; four distinct practices with rationale.

A7. *Model answer.* Architecture means accessibility is a *property of the structure*: design-system primitives implement correct semantics, keyboard behavior, focus management, and labelling so product code inherits them; page/route structure includes landmarks, headings, skip links, and route-change focus/announcement; forms and error handling are centralized (error summaries, `aria-describedby` wiring); acceptance criteria and PR checklists include a11y; and tests run in CI at the component level. Mechanisms (any three): accessible primitives/headless components; lint rules (`jsx-a11y`) and automated checks (axe) in CI; focus management in router/layout; tokens ensuring contrast; content guidelines. Automation cannot verify: whether the accessible name is *meaningful*, whether the reading/focus order makes sense, whether the interaction is usable with a screen reader, cognitive load, or real assistive-technology behavior; manual keyboard and screen-reader testing remain necessary. *Must-haves:* "inherit by construction" idea; three mechanisms; limits of automation.

A8. *Model answer.* (1) Word order and grammar vary: concatenated fragments (`"You have " + n + " items"`) cannot be reordered by translators → use whole messages with placeholders (ICU MessageFormat via a library such as FormatJS or i18next with an ICU plugin; confirm your chosen library's ICU support). (2) Plural rules differ (some languages have 1, 2, or 6 categories) → ICU plural selectors / `Intl.PluralRules`. (3) Number, currency, date formats differ → `Intl.NumberFormat`, `Intl.DateTimeFormat` with explicit locale and time zone; never parse display strings. (4) Direction: RTL scripts need mirrored layout → `dir` / `lang` attributes, CSS logical properties (`margin-inline-start`), bidi-safe components. (5) Text length expansion and fonts → flexible layouts, no fixed-width text

containers; (6) Sorting/collation → `Intl.Collator`. Any four with mechanisms. *Must-haves:* four distinct problems paired with real mechanisms.

A9. *Model answer.* Retries multiply load on a service that is already struggling (retry storms/thundering herd), and if every client retries synchronously with the same delay, the retries arrive in waves; they also duplicate non-idempotent side effects (double charge). Safe policy: retry only idempotent operations (GET, or POST with an idempotency key) and only for transient failures (network errors, 408/429/502/503/504, honoring `Retry-After`); exponential backoff with jitter (random delay to desynchronize); a small attempt cap and an overall deadline/timeout budget; per-attempt timeouts; abort on user navigation/cancel; circuit breaking or load shedding when failures persist; surface a fallback UI after the budget is spent. *Must-haves:* storm mechanism; idempotency restriction; backoff+jitter+cap/deadline.

A10. *Model answer.* (a) An ADR records what was decided and why at a point in time; editing it destroys the history that lets the team learn and audit. A changed decision is recorded as a new ADR that supersedes the old one; the old ADR's status line changes to "Superseded by ADR-NNNN"; only typos, links, and clarifications that do not change the decision are edited. (b) Tutorial (learning, guided), how-to (task for the competent), reference (facts), explanation (understanding/rationale). A runbook is a how-to (task under pressure); an ADR is explanation (rationale). *Must-haves:* immutability + supersession; four types; correct classification.

Section B

B1. Stage 1 (30%): buckets < 30 → `ana` (12). Stage 2 (65%): `ana`, `ben` (37), `cho` (64). Stage 3 (0% with allow `dev`): `dev` only; `ana`, `ben`, `cho` are off because `bucket < 0` is false (this is an *explicit change of rule*; it models turning the feature off for everyone but an allow-listed user). Stage 4 (100%): all four (`bucket < 100`). (a) Percentage increases (1→2, and 2→4) never remove users because the enabled set is `{bucket < p}`, which grows monotonically in `p`. The stage 2→3 change *does* remove users, but that is a deliberate rule change (kill/rollback), not a ramp. (b) Yes: `ana`'s bucket for `y` is independent (e.g., 88) because the hash input includes the flag key. Salting prevents the same low-bucket users from being the early cohort for every flag (cohort correlation), which would give a small group all of the risk and bias experiments. *Scoring:* 1 pt for all four stage sets (allow one slip), 1 pt for (a)+(b) reasoning.

B2. 1. `DEFAULTS` = `{v:2, density:'comfortable', theme:'system'}`. 2. `{v:2, density:'compact', theme:'system'}`. 3. `V1` fails (`v` literal mismatch); `V2` succeeds with `theme` defaulted → `{v:2, density:'compact', theme:'system'}`. 4. Neither schema matches (`v: 3`) → `DEFAULTS`. 5. `V2` fails (`density` invalid) → `DEFAULTS`. 6. `JSON.parse` throws → `catch` → `DEFAULTS`. A user on the newer version who then rolls back to this code has their saved preferences silently reset to defaults (the data is not corrupted *unless* the old code later writes defaults over the `v:3` blob, destroying it). Remedies: keep new formats readable by older code during a rollback window (expand/contract), write both formats, or preserve unknown keys/versions rather than overwriting. *Scoring:* 1.5 pts for six outputs (–0.25 per error, min 0), 0.5 pt for rollback consequence.

B3. (a) A normal reload at 10:45: `index.html` is requested with a conditional request (`If-None-Match`); the server returns `200` with new content (ETag changed) referencing `app.BBB111.js`; `app.BBB111.js` is requested (not cached) → 200; `app.AAA111.js` is *not* requested (HTML no longer references it). `dash.BBB222.js` is requested when the dashboard is opened. (b) At 10:30 the stale tab's runtime requests `dash.AAA222.js`; the file no longer exists, the SPA fallback returns `index.html` with status 200 and `text/html`; the browser tries to evaluate/parse the module and fails (for modules: MIME type error

"Expected a JavaScript module script but the server responded with a MIME type of text/html"; for classic scripts: `Unexpected token '<'`); React lazy import rejects with a chunk-load error → if no error boundary/handling, blank page. (c) Any two: retain old hashed assets for a window (≥ longest client lifetime, e.g., 14 days); exclude `/assets/` from the SPA fallback (return real 404s); handle dynamic-import failure with a boundary that prompts "new version available — reload"; poll `version.json` and show an update banner; deploy assets before HTML and avoid deleting. *Scoring:* 0.7 for (a), 0.7 for (b), 0.6 for (c).

B4. `mode="render"`: initially the fallback `Something broke` (the render throws on first render, boundary catches); click not possible. Error surfaces to the boundary (`componentDidCatch` / `getDerivedStateFromError`) and, in React 19, root callback `onCaughtError`. `mode="click"`: initially the button `go 0`; after click the handler throws before `setN`, so the UI is unchanged (`go 0`); not caught by the boundary; surfaces to `window.onerror` (React event handler errors are reported as uncaught exceptions via the global error mechanism; sandbox-confirmed with react-dom 19.3.1: `window` `error` fired, no root callback ran, and the tree stayed mounted). `mode="timer"`: initially `go 0`; click schedules the timer and calls `setN(1)` → renders `go 1`; later the timeout throws an uncaught error → `window.onerror`; UI unaffected. `mode="promise"`: initially `go 0`; click creates a rejected promise (nothing handles it) and `setN(1)` runs → `go 1`; the rejection surfaces as `unhandledrejection`. For `mode="render"` without a boundary: React unmounts the whole root → blank page (any static HTML outside the root remains); the error is reported via `window.onerror` / React 19 `onUncaughtError`. *Scoring:* 0.4 per mode (render, click, timer, promise) with 0.4 for the no-boundary case.

B5. 1. Not breaking: input widened (`Item[]` is assignable to `readonly Item[]`). 2. Not breaking: consumers' `(id) => ...` is still assignable to `(id: string, item: Item) => void` (a function taking fewer parameters can be used where more are supplied); the new arg is additive. 3. Breaking: output widened; the consumer's `switch` now has an unhandled `'error'` case, so `assertNever` in `default` fails to compile (it receives `'error'`, not `never`) and runtime behavior would hit the assertion. 4. Breaking: optional → required input; existing consumers who omit `dense` get a compile error (and JS consumers get `undefined`). *Scoring:* 0.5 each; require reasoning for credit.

Section C

C1 (8 points). Model solution:

```tsx
import type { ComponentPropsWithRef } from 'react';

type Tone = 'primary' | 'secondary' | 'ghost';

type ButtonProps = ComponentPropsWithRef<'button'> & {
  tone?: Tone;
  /** @deprecated Use `tone`. `'link'` maps to `'ghost'`. Removed in v5. */
  variant?: Tone | 'link';
};

const warned = new Set<string>();
function warnOnce(key: string, message: string) {
  if (process.env.NODE_ENV === 'production' || warned.has(key)) return;
  warned.add(key);
  console.warn(`[@acme/ds] ${message}`);
}

export function Button({ tone, variant, ref, ...rest }: ButtonProps) {
  if (variant !== undefined) warnOnce('Button.variant', '`variant` is deprecated; use `tone`.
It will be removed in v5.');
  const legacy: Tone | undefined = variant === 'link' ? 'ghost' : variant;
  const resolved: Tone = tone ?? legacy ?? 'primary';
  return <button ref={ref} data-tone={resolved} {...rest} />;
}
```

Evaluation: *(2)* both props typed, `variant` documented as `@deprecated`; *(2)* precedence `tone ?? mapped variant ?? default`, `'link'` → `'ghost'`; *(1)* `ref` as a normal prop (React 19) with correct typing; *(1)* warning once, dev-only, no production warning; *(1)* version: MINOR (backward-compatible addition; deprecation is not a break); *(1)* contract: removing `variant` is a MAJOR release, only after migration (codemod, usage data near zero, deprecation period, changelog/migration guide). Deductions: using `forwardRef` unnecessarily is acceptable only with a note that the support range includes React 18 (then correct); missing `Omit`/conflicts are not penalized. Common errors: letting `variant` win; warning on every render; classifying as PATCH.

C2 (6 points). Model solution (as an HTTP response header; line breaks for readability only):

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' https://analytics.example;
  style-src 'self';
  img-src 'self' data: https://img.example;
  connect-src 'self' https://api.example.com;
  object-src 'none';
  base-uri 'self';
  frame-ancestors 'none';
  form-action 'self'
```

Explanations: `default-src` is the fallback for unlisted fetch directives (restrictive baseline); `script-src` allows same-origin bundles and the single analytics origin (prefer a path or hash/nonce-based allowance if analytics supports it; host allowlists are weaker than nonces); no `'unsafe-inline'` / `'unsafe-eval'` because there are no inline scripts and bundles don't need `eval` (production builds); `style-src 'self'` as there are no inline styles; `img-src` includes `data:` and the image CDN; `connect-src` governs fetch/XHR/WebSocket destinations; `object-src 'none'` blocks plugins; `base-uri 'self'` prevents `<base>` hijacking; `frame-ancestors 'none'` forbids framing (clickjacking defense; effective only via the HTTP header, not `<meta>`); `form-action 'self'` limits form submission targets. Rollout: first deploy as `Content-Security-Policy-Report-Only` with a reporting endpoint (`report-uri` is deprecated but widely supported; `report-to` needs a `Reporting-Endpoints` header and is newer, Firefox 149+, so send both during the transition), monitor violations across real traffic (including extensions noise), fix, then enforce; keep reports in observability; review on each third-party addition. If a runtime CSS-in-JS library that injects `<style>` elements is introduced: you either need `style-src` with a nonce passed to the library (preferred where supported), or `'unsafe-inline'` (weakens style injection protections), or switch to extracted CSS; note the trade-off. Also note that the inline JSON config `<script type="application/json">` is data and not executed, so it needs no allowance, whereas an inline JS config assignment would need a nonce/hash. Evaluation: *(3)* correct directives for each fact; *(1)* no unsafe-inline/eval; *(1)* report-only rollout; *(1)* CSS-in-JS consequence. Wrong: `default-src *`, `'unsafe-inline'` for scripts, `frame-ancestors` via meta.

C3 (6 points). Model solution:

```ts
export class HttpError extends Error {
  constructor(public status: number, message: string, public body?: unknown) { super(message); this.name = 'HttpError'; }
}

interface Options extends Omit<RequestInit, 'signal'> {
  signal?: AbortSignal;           // caller's cancellation
  timeoutMs?: number;             // per attempt
  maxAttempts?: number;           // total attempts (default 3)
  baseDelayMs?: number;           // backoff base
  maxDelayMs?: number;
}

const IDEMPOTENT = new Set(['GET', 'HEAD', 'OPTIONS', 'PUT', 'DELETE']);
const RETRYABLE_STATUS = new Set([408, 429, 502, 503, 504]);

const sleep = (ms: number, signal?: AbortSignal) =>
  new Promise<void>((resolve, reject) => {
    if (signal?.aborted) return reject(signal.reason ?? new DOMException('Aborted', 'AbortError'));
    const id = setTimeout(() => { signal?.removeEventListener('abort', onAbort); resolve(); }, ms);
    const onAbort = () => { clearTimeout(id); reject(signal!.reason ?? new DOMException('Aborted', 'AbortError')); };
    signal?.addEventListener('abort', onAbort, { once: true });
  });

export async function fetchJson<T>(url: string, opts: Options = {}): Promise<T> {
  const { signal, timeoutMs = 8000, maxAttempts = 3, baseDelayMs = 300, maxDelayMs = 5000, ...init } = opts;
  const method = (init.method ?? 'GET').toUpperCase();
  const canRetry = IDEMPOTENT.has(method);

  for (let attempt = 1; ; attempt++) {
    const controller = new AbortController();
    const onCallerAbort = () => controller.abort(signal!.reason);
    if (signal?.aborted) throw signal.reason ?? new DOMException('Aborted', 'AbortError');
    signal?.addEventListener('abort', onCallerAbort, { once: true });
    const timer = setTimeout(() => controller.abort(new DOMException('Timeout', 'TimeoutError')), timeoutMs);

    try {
      const res = await fetch(url, { ...init, signal: controller.signal });
      if (res.ok) return (await res.json()) as T;

      const retryAfter = Number(res.headers.get('retry-after'));
```

```
      const err = new HttpError(res.status, `HTTP ${res.status}`);
      if (!canRetry || !RETRYABLE_STATUS.has(res.status) || attempt >= maxAttempts) throw err;
      const backoff = Math.random() * Math.min(maxDelayMs, baseDelayMs * 2 ** (attempt - 1));
// full jitter
      const delay = Number.isFinite(retryAfter) && retryAfter > 0 ? Math.min(retryAfter *
1000, maxDelayMs) : backoff;
      await sleep(delay, signal);
    } catch (e) {
      if (signal?.aborted) throw e;                              // caller cancelled: never
retry
      const isTimeoutOrNetwork = !(e instanceof HttpError);      // fetch rejects with
TypeError on network failure; our timeout abort too
      if (!canRetry || !isTimeoutOrNetwork || attempt >= maxAttempts) throw e;
      await sleep(Math.random() * Math.min(maxDelayMs, baseDelayMs * 2 ** (attempt - 1)),
signal);
    } finally {
      clearTimeout(timer);
      signal?.removeEventListener('abort', onCallerAbort);
    }
  }
}
```

Evaluation: *(1)* per-attempt timeout via own `AbortController` + `setTimeout` (no unsupported APIs); *(1)* caller abort honored (propagated, never retried, listeners cleaned); *(1)* retry restricted to idempotent + retryable statuses and network errors, attempts capped; *(1)* exponential backoff **with jitter** (random in `[0, cap]`); *(1)* `Retry-After` seconds honored (bounded); *(1)* `HttpError` with `status`, timers/listeners cleaned in `finally`. Subtle points to credit: a thrown `HttpError` inside `try` must not be retried by the `catch` (it checks `instanceof HttpError`); the `finally` runs on each attempt; `Retry-After` can also be an HTTP date (acknowledged limitation). Common errors: retrying POST; fixed delay without jitter; leaking timers; swallowing caller aborts; retrying 4xx like 400/401/404.

C4 (5 points). Model ADR (abridged but complete; yours may differ if the reasoning is sound):

```
# ADR-0027: Keep the session in an httpOnly cookie via a BFF; no access tokens in localStorage

- Status: Accepted (2026-06-10)
- Deciders: Web architecture group; Security; Identity team
- Tags: security, auth

## Context
The SPA stores a 15-minute access token and a 30-day refresh token in `localStorage`. A June security review
found 3 locations where user-authored HTML reaches `dangerouslySetInnerHTML` (2 fixed, 1 requires a product change), and our
dependency audit shows 412 transitive packages in the production bundle. Any XSS can currently exfiltrate refresh tokens
and silently persist access for 30 days. Support handled 9 account-takeover reports last quarter (2 attributed to token theft).
A BFF already exists for SSR of marketing pages (3 engineers on-call). 94% of traffic is same-site (`app.example.com` and `api.example.com`).

## Decision drivers
1. Limit blast radius of XSS (tokens must not be readable by page scripts).
2. Keep logins seamless for existing users; no forced re-login storm.
3. Operational cost and team capacity.
4. CSRF risk must remain controlled.

## Considered options
1. **BFF-managed session in an httpOnly, Secure, SameSite=Lax cookie; BFF holds/refreshes tokens** (chosen).
2. Keep tokens in `localStorage`, add CSP and sanitization hardening only.
3. Keep tokens in memory only (refresh via silent iframe/cookie).
4. Move to a third-party hosted auth UI/session.

## Decision
We will terminate sessions at the BFF: the browser holds only an opaque, httpOnly, Secure, SameSite=Lax session cookie; the BFF stores/refreshes
tokens server-side and proxies API calls. State-changing requests require a custom header (`X-CSRF`) verified by the BFF plus Origin checks.
The SPA will not store credentials in web storage. Applies to the customer app; the admin app follows in Q4.

## Consequences
+ Page scripts (including injected ones) cannot read credentials; XSS impact is reduced (though not eliminated: an attacker can still perform actions as the user while the page is open).
+ Refresh token rotation and revocation are centralized.
- BFF becomes a critical path: availability and latency requirements rise (+ ~10–20 ms per API call; on-call load).
```

```
- CSRF must now be handled explicitly (cookies are ambient credentials); mistakes here are new risk.
- Session storage on the server needs a store (Redis) with failure modes (session loss on outage).
- Migration complexity: running both mechanisms during rollout; existing tokens must be exchanged on next visit.
## Validation and enforcement
- Lint rule forbids `localStorage.setItem` with keys matching `/token|jwt|session/i`; CI e2e asserts no credential in web storage.
- Pen test of CSRF protections before 100% rollout; dashboards: login success rate, 401 rate, BFF p95 latency.
## Revisit trigger
If BFF p95 latency > 150 ms for 2 weeks, or if cross-site embedding requirements (third-party embeds) require SameSite=None.
```

Evaluation: *(1)* context with facts/numbers and the actual problem; *(1)* ≥3 genuine options including status quo/hardening; *(1)* specific decision with scope; *(1)* honest negatives (CSRF, availability, complexity) not only upsides; *(1)* enforcement/validation + revisit trigger. Penalize: "XSS can't happen anymore" (overclaim); no CSRF discussion; options that are strawmen.

Section D

D1 (5 points). *Findings:* (1) HTML is cacheable for a day (`max-age=86400`) at the CDN (`x-cache: HIT`, `age: 41210` ≈ 11.4 h): the edge serves the old `index.html` (ETag `7c1f` stale) without contacting the origin; clients also cache it; so users receive HTML that references old asset hashes (and `version.json` shows the new build, proving the origin is correct and caching is the issue; note `version.json` is itself potentially cached; here it returned the new build, so it is not cached long). (2) Hashed assets have `max-age=300`: wasteful (revalidation/re-download every 5 minutes), they should be immutable year-long. (3) A hard refresh bypasses caches, explaining why it fixes the issue. *Fix:* `Cache-Control: no-cache` (or `max-age=0, must-revalidate`) for HTML at origin and CDN configuration respecting it (ensure CDN doesn't override with its default TTL; set edge TTL for HTML to 0 or use short `s-maxage` with purge on deploy); `public, max-age=31536000, immutable` for hashed assets; `no-store` for `version.json`. *Immediate impact:* purge/invalidate the CDN cache for `/` and `/index.html` (and any route fallbacks); older browser-cached copies expire by the 24 h TTL: mitigate with the update banner driven by `version.json` and, if possible, a one-time service worker/`Clear-Site-Data` strategy only if justified; ensure old assets are still available (stale HTML references old hashes). *Verification:* `curl -I` on `/` shows `no-cache` and `x-cache: MISS/REVALIDATED` after deploy; a deploy verification step compares served build ID from several regions; add a test in the pipeline to assert headers; monitor the percentage of sessions by build ID after deploy (should converge within minutes). *Scoring:* 1.5 diagnose HTML caching as primary cause with evidence; 1 secondary asset caching issue; 1.5 fixes (headers + CDN purge + update banner); 1 verification.

D2 (5 points). *Findings and fixes:*

1. **Stored XSS** via `dangerouslySetInnerHTML` with user-authored HTML: attacker note includes `<img src=x onerror=...>` that runs in the victim's origin, stealing the `localStorage` token or performing transfers. Fix: render as text; if rich text is required, sanitize with a maintained sanitizer

(DOMPurify) configured with an allowlist, or render from a structured format (Markdown AST) rather than raw HTML; add CSP (no inline script/handlers) as defense in depth; sanitize on the server as well.

2. **Unsafe URL in** `href` (`javascript:` or `data:` URLs from `website`): clicking executes script (React warns about `javascript:` URLs in development but don't rely on that). Fix: parse with `new URL(website, base)` and allow only `http:` / `https:`; add `rel="noopener noreferrer"` with `target="_blank"` (modern browsers imply `noopener` for `_blank`, but be explicit for older ones; `noreferrer` also prevents referrer leakage).
3. **Token in** `localStorage`: readable by any XSS; long lifetime increases impact. Fix: httpOnly Secure SameSite cookie via BFF (see C4) or in-memory short-lived tokens with rotation.
4. **Login request over** `fetch` **without** `credentials` **and with default content type** `text/plain;charset=UTF-8` for a JSON string body: server must not parse by content type blindly; the login not using `Content-Type: application/json` is a correctness/CSRF-hardening issue (login CSRF). Fix: set `content-type: application/json` explicitly (which forces a CORS preflight for cross-origin requests), and validate content type server-side.
5. **CSRF on** `/api/transfer`: because the server accepts cookie sessions, a malicious site can submit a cross-site request with the victim's cookies; the missing `content-type` makes the body `text/plain`, a "simple" request type that doesn't trigger a preflight, so a server that parses JSON regardless of content type is exploitable. Fix: `SameSite=Lax/Strict` cookies; require a custom header (e.g., `X-CSRF-Token`) or token verified server-side; require `application/json` and reject others; check `Origin` / `Referer`; require re-authentication/step-up for transfers; do not accept both cookie and bearer auth without clear rules.
6. **No input validation / money as** `number` (design smell): amounts as floating-point; server must validate; use minor units and idempotency keys (`Idempotency-Key`) to prevent double submissions. *Scoring:* 1 pt each for XSS, URL, token storage, CSRF (with exploit explanation), plus 1 pt for fixes quality/other (login content type, idempotency, CSP). Deduct if only generic advice ("sanitize everything") is given.

D3 (5 points). (a) *Discriminating:* flag on/off (1,897 vs 21) ties errors to the new feature; locale (`de-DE` 1,203, `tr-TR` 689 vs `en-US` 0) is the strong discriminator. *Noise:* release (almost all traffic is on 3.8.0 now, and the flag explains it), browser (spread across browsers proportional to usage; not discriminating). (b) *Root cause:* `new Date(input)` is given a locale-formatted *display* string (`'15.03.2026'`); `Date` parsing of non-ISO strings is implementation-defined and typically yields Invalid Date for `dd.MM.yyyy`; `toISOString()` on an Invalid Date throws `RangeError: Invalid time value`. `en-US` strings like `'3/15/2026'` are parsed by most engines as month/day/year (a non-standard but widely implemented fallback), so `en-US` appears to work (and `en-GB`'s `'15/03/2026'` often fails, but few `en-GB` sessions reached the step or the number is low: 2 errors). Even in `en-US` it is a latent bug (day/month ambiguity: `'03/04/2026'` could be misparsed in other locales, producing *wrong dates silently*, which is worse than an exception). (c) *Mitigation:* set the flag `checkout.delivery-date` to 0% (fastest, smallest blast radius, no redeploy; the 20% cohort), then verify error sessions drop and funnel recovers; consider wrapping the step in an error boundary with a fallback to the previous flow. (d) *Permanent fix:* never parse display strings; keep the picker's *value* as ISO `YYYY-MM-DD` (or separate year/month/day numbers, or a `Date` constructed from components) and use `Intl.DateTimeFormat(locale, { timeZone })` for display only; parse user typed text with an explicit-format parser (e.g., date-fns `parse` with a pattern) and validate; `toApiDate` should accept a typed value, not a display string. Guardrails: tests parameterized across locales (`de-DE`, `tr-TR`, `en-US`, `ar-AE`) and across time zones with fixed clock; property test round trip `format→parse`; lint rule against `new Date(string)` with non-literal arguments (e.g., `no-restricted-`

`syntax`); detection: locale as a first-class dimension on error reports and canary gate (cohort comparison per locale), plus a per-locale synthetic checkout run. (e) *Missing dimension:* none was missing in this table (locale was present, which made diagnosis fast); credit also for noting that *browser* locale vs *app* locale vs *format* locale should be recorded distinctly, and for time zone if date-handling. *Scoring:* 1 (a), 1.5 (b), 0.75 (c), 1.25 (d with two guardrails), 0.5 (e).

Section E

E1 (7 points). *Model answer. Versioning.* (1) Token rename: **MINOR** release adding `color.text.primary` (and CSS custom property alias) while keeping `color.text.default` as a deprecated alias; removal in the next **MAJOR**. (2) `tone`: **MINOR** expand release (both props, tone wins, dev warning, `@deprecated`); removal of `variant` in the same next MAJOR. (3) React peer bump to ≥19: **MAJOR** (operational breaking change), bundled into that same v5.0.0 to limit the number of majors consumers face, but *separately sequenced in time*: apps first upgrade to React 19 (using the 18.3 warnings clean-up) before taking v5, or ship v4.x supporting `^18.3 || ^19` first so ds upgrade and React upgrade are independent steps. *Sequence.* Weeks 1–2: release v4.5 (expand) with aliases/dual props/peer range `^18.3.0 || ^19.0.0`; publish codemods (token rename for CSS/TS/theme objects; `variant` → `tone` jscodeshift/ts-morph, with manual-review report for dynamic cases) and migration guide; add lint rules (warn) in the shared ESLint plugin; scanning job over all 30 repos producing burndown (usage count per app per deprecated API and React version). Weeks 2–8: apps run codemods; automated PRs from a bot for each repo (Renovate/custom) with CI evidence; office hours; teams migrate React to 19 on their own schedule (needs v4.5 compat). Week 6: lint rules to error; announce removal date. Weeks 8–10: chase laggards using data: for each, an owner and a date; for unmovable apps, offer a compatibility shim package or pin to v4.x with a documented support end date (security fixes only). Week 11–12: when usage is zero (or signed-off exceptions), release v5.0.0 (removal + React 19 peer). Rollback: because v4.5 is purely additive and v5 is opt-in per app, rollback is "pin previous version"; the token alias means no CSS rewrite is required to roll back; keep v4.x security-patch branch for N months. *Communication & measurement.* Single announcement + dated timeline, migration guide, changelog; weekly burndown dashboard; per-team status; blockers tracked; success metric: % of apps with zero deprecated usage; time-to-upgrade; incidents attributed to migration. *What NOT to do:* big-bang v5 with all three changes without an expand phase; remove deprecated aliases on a fixed date regardless of usage data; rename tokens silently in a patch; require all teams to upgrade React simultaneously; depend on emailed instructions without tooling; leave the deprecated code forever (set removal gate). *Scoring:* 2 for correct version classification and expand/contract; 2 for tooling + automation + measurement; 1 for laggards plan; 1 for communication/rollback; 1 for explicit anti-patterns/trade-offs. Alternative sequencing is acceptable if reasoned.

E2 (6 points). *Model answer. Ownership:* map each package/directory to exactly one owning team via CODEOWNERS and package metadata (owner, on-call, Slack); `shared/utils` is dismantled: domain code moves to the owning feature/domain package; truly generic utilities go to a small `@acme/platform-utils` with a platform owner and a public API; rule of three before adding to shared. *Boundaries:* features expose a public entry point only; enforce with lint (`no-restricted-imports`/boundaries plugin) + dependency analysis in CI (no cycles, no feature→feature internals, shared never imports features); package `exports` for packages that are real workspaces. *Merge/review mechanics:* merge queue (tests on rebased PR with queue ahead) to cure semantic-conflict red mains; affected-only CI and caching to keep p90 under ~10 min; PR size norms and review SLA; revert-first on red main; flaky test quarantine with an owner and deadline. *Debt:* hotspot analysis (churn × complexity) and incident tags to choose the

top three debts; a register with interest/principal estimates; capacity reservation (e.g., ~15% as a starting heuristic, adjusted by data); ratchets for new violations (e.g., imports of old `shared/utils`, `any`, lint disables); flag expiry registry. *Decision recording:* ADR log in the repo with a template and index; an ADR for each significant existing but unexplained choice recorded retroactively as "Accepted (recorded retrospectively)" with the best-known context; RFC for cross-team changes; link ADR IDs in lint messages. *Sequencing:* (week 1–2) CODEOWNERS + merge queue + red-main metrics (quick wins that reduce pain); (weeks 2–6) boundary lint in warn mode with a violations baseline and ratchet; start `shared/utils` split by moving the most-imported domain pieces; ADR template and retrospective ADRs for top 5 mysteries; (weeks 6–12) boundary rules to error for new code; continue decomposition opportunistically and via debt capacity; CI speedups. *Targets:* red main < 1 hour/week; PR time-to-first-review < 1 business day; CI p90 < 10 min; cross-team PR share of shared files down 50%; zero new imports of deprecated shared paths; 100% packages with owner metadata; ADRs for all new significant decisions. Teams keep delivering because each step is incremental and behavior-neutral. *Scoring:* 1.5 ownership+boundaries; 1.5 merge/review/CI mechanics; 1 debt; 1 ADR/knowledge; 1 sequencing + measurable targets.

E3 (6 points). *Model answer. Principles:* auth is high-impact and hard to roll back (session invalidation), so use dual-running, small cohorts, and fast levers. *What is flagged:* a server-side/BFF-controlled switch (flag evaluated server-side or in an edge/BFF layer so security decisions are not client-controlled) selecting the identity provider per cohort; client flag only toggles UI entry points. Support both providers' tokens in the backend (dual validation) for the migration window. *Cohorts and stages:* internal staff (100%) → opted-in beta customers → 1% new logins (new sessions only, sticky by user ID) → 5% → 25% → 50% → 100%; existing sessions continue on the old provider until expiry or silent migration at next login/refresh. Avoid mid-session switching. *Instrumentation before rollout:* login funnel metrics by provider (start → credentials submitted → MFA → success), login success rate, time-to-login p50/p95, token refresh success, 401/403 rates, redirect-loop detectors (more than N auth redirects in M seconds client-side), support ticket volume tags, error sessions by provider/browser/locale/flag/release, synthetic login checks per provider and per region, trace IDs through BFF to IdP. Release tags and flag variant on all events. *Abort criteria:* login success rate for new-provider cohort worse than control by > X points (with minimum sample thresholds), p95 login latency > 1.5× baseline, any security anomaly, spikes in lockouts/support tickets; automated kill (flag to old provider) with the old path kept warm. *Rollback:* provider flag flips new logins back; sessions created by the new provider must remain valid for at least a rollback window or be silently re-authenticated; keep the dual validation until after 100% + soak; stale clients: server tolerates both client versions; clients poll config to learn of provider changes; ensure token/cookie names don't collide. *Communication:* status page and support runbook with known symptoms and workarounds; customer comms for password/MFA re-enrollment if needed. *Rehearsals:* game day in staging: provider failover, flag flip mid-load, token expiry mid-session; load test the new IdP; test account recovery flows; run a rollback drill before 1% production. *Post-rollout:* remove the old provider path (cleanup ticket with date), retire flags, ADR recording the decision and revisit conditions. *Scoring:* 1.5 rollout/cohort design; 1.5 instrumentation and abort criteria (specific); 1.5 rollback including stale clients/sessions; 1 rehearsal and communication; 0.5 cleanup/ADR. Penalize generic "use a canary" without specifics and neglecting sessions/dual validation.

E4 (6 points). *Model answer. i18n:* externalize all strings with ICU messages and plural rules (German plural fine; Arabic has six plural categories); locale resolution (user setting > URL > `Accept-Language`); `Intl` formatting for numbers/dates/currencies (EUR, AED) with explicit locale and time zone; catalog loading by locale (code split); pseudo-localization in CI; translation workflow with context and

screenshots; fonts supporting Arabic; text expansion (German is long) tested in layouts; sorting with `Intl.Collator`; no date/number parsing from display strings (see D3). *RTL:* `dir="rtl"` and `lang="ar"` on the document, CSS logical properties instead of left/right, mirrored icons (directional only), bidirectional text handling for mixed content (numbers, Latin brand names), RTL-aware components in the design system (menus, carousels, sliders, focus order, keyboard arrow semantics), visual regression for RTL stories. *Accessibility:* language attributes for screen readers, correct focus order in RTL, contrast and tap targets across scripts, announcements localized; test with Arabic screen-reader voices ⚠ (verify coverage); since a11y is architectural (08), fix in primitives. *Security/privacy:* GDPR/ePrivacy expectations for Germany: consent management before non-essential cookies/storage/telemetry; classify what the client stores (localStorage, IndexedDB, cookies) and which telemetry (session replay, analytics) needs consent; data residency considerations ⚠ (legal review); minimize PII in logs and error reports; CSP review for new third parties (payment, analytics per market); UAE-specific legal/regulatory requirements for data handling ⚠ (verify with legal). *Resilience:* CDN/regional performance (latency to Europe/Middle East), fallback for locale catalogs failing to load (default to English with notice), payment provider differences per market with degraded modes, timeouts/retries tuned for higher RTT, offline/poor network behavior. *Governance/DS:* design system must support logical properties/RTL and locale-dependent typography; a contribution/review checklist includes RTL/i18n/a11y; tokens for typography per script; ADR for i18n library and message format; docs for localization workflows; versioning of message catalogs and backward compatibility of keys. *Verification before launch:* pseudo-localization and RTL visual regression, e2e checkout in `de-DE` and `ar-AE` with real formats, a11y audit with native speakers/users, performance budgets per region, consent flows verified (no non-essential storage before consent), legal sign-offs, synthetic monitoring from the target regions, dashboards with locale as a dimension, rollout by market with flags. *Cost of future markets:* locale-agnostic components, centralized formatting utilities, translation pipeline automation, market config as data (currency, address formats, legal text), regression suites parameterized by locale, and ADR/documentation capturing the playbook. *Scoring:* 1.5 i18n+RTL; 1 accessibility; 1.5 privacy/security with consent and storage/telemetry; 1 resilience/performance; 1 governance/DS + verification and future-proofing. Prioritization explicit (what blocks launch vs follow-up) is required for full credit.

Part 3 — Scoring rubric

3.1 Point allocation summary

Section	Questions	Points	Category
A	A1–A10 (2.5 each)	25	Explanation
B	B1–B5 (2 each)	10	Debugging
C	C1 (8), C2 (6), C3 (6), C4 (5)	25	Implementation
D	D1–D3 (5 each)	15	Debugging
E	E1 (7), E2 (6), E3 (6), E4 (6)	25	Reasoning
Total		**100**	

3.2 Partial credit guidance

- **Explanation (A):** 2.5 = all three must-haves present with correct causal reasoning; 1.5–2 = two must-haves or all three with minor errors; 0.5–1 = one must-have or correct but purely keyword-level; 0 = incorrect/absent. Statements that are confidently wrong about security or correctness lose points even if other parts are right.
- **Prediction (B):** points per the breakdown in each answer; reasoning is required; a correct final answer with incorrect reasoning receives at most half credit.
- **Implementation (C):** code must be *plausibly correct* (compiles in principle, correct semantics). Award points per the evaluation list; subtract for bugs that would cause incorrect behavior (e.g., retrying non-idempotent calls, leaks, unsafe CSP). Style choices are not penalized.
- **Debugging (D):** credit evidence-based reasoning; solutions that list fixes without tying them to evidence receive at most 60% of the points.
- **Reasoning (E):** use the listed scoring criteria; credit alternative designs that are internally consistent and address constraints. Do not award points for buzzwords without trade-offs. Naming a negative consequence or limitation of your own design is rewarded.

3.3 Category computation

```
Explanation%     = (sum of A scores) / 25 × 100
Implementation%  = (sum of C scores) / 25 × 100
Debugging%       = (sum of B + D scores) / 25 × 100
Reasoning%       = (sum of E scores) / 25 × 100
Overall%         = total / 100 × 100
```

3.4 Pass thresholds

Requirement	Threshold
Overall	≥ 80 / 100
Explanation	≥ 17.5 / 25 (70%)
Implementation	≥ 17.5 / 25 (70%)
Debugging	≥ 17.5 / 25 (70%)
Reasoning	≥ 17.5 / 25 (70%)

You must meet **all five** thresholds.

3.5 Interpretation bands

Overall	Interpretation
90–100	Strong professional command; ready to lead production engineering practices on a team
80–89	Solid; meets the bar; review missed items
70–79	Partial: you understand most concepts but have gaps in execution or reasoning; remediate and retake
< 70	Revisit the phase lessons (see remediation map) before retaking

---

Part 4 — Promotion criteria

You may proceed to Phase 11 — Senior and architect mastery when **all** are true:

1. **Scores:** overall ≥ 80% and each category ≥ 70% (Part 3.4).
2. **Evidence of practice (recommended, not scored):**
   - You have written at least one real ADR for a decision in a codebase you work on (or the project assigned in your cohort).
   - You have walked through one runbook in a simulated failure (game day) and recorded what you would fix.
   - You have implemented a deploy-safe asset caching policy and verified it with `curl -I`.
   - You have run a dependency update and read the diff of at least one non-trivial upgrade.
3. **Self-explanation check:** without notes, explain aloud (to a peer or recording) in under five minutes: (a) the difference between deploy and release and how you would roll back a bad change in your own app; (b) one incident in the style of INC-2417: signals, segmentation, mitigation, root cause, contributing factors.

If you did not meet the thresholds: remediation map

Weak area (category)	Review these lessons	Then do
Explanation of delivery/config/observability (A2, A3, A4)	02, 03, 04, 05, 18	Re-answer A2–A4 aloud; draw the deployment pipeline for your app
Security explanations/debugging (A5, A6, C2, D2)	06, 07	Perform a mini threat model on a sample app; write a CSP in report-only mode
A11y/i18n/resilience (A7–A9, C3, E4)	08, 09, 10	Add pseudo-localization and RTL stories to a component; re-implement C3
Compatibility/migrations (B2, B5, C1, E1)	11, 12, 13	Write an expand/contract plan for a real rename; write a codemod
Organization/debt/decisions (A1, A10, C4, E2)	01, 14, 15, 16, 17	Produce a debt register and two ADRs for your codebase
Incident skills (B3, B4, D1, D3, E3)	18, 19	Re-read the INC-2417 worked incident and write your own timeline for a fictional incident

Retake after at least one week using the *same* questions with fresh answers *and* a new scenario of your own for each of D3 and E3 (to avoid memorizing the key).

Part 5 — Instructor / self-review notes

- **Facts to verify when reviewing:** React 19 root error options (`onCaughtError`, `onUncaughtError`, `onRecoverableError`) and their exact semantics; DOMPurify as a well-known sanitizer; browser support dates for `Array.prototype.toSorted` (Safari 16, Chrome 110, per MDN/caniuse); `Content-Security-Policy` directive behavior, especially that `frame-ancestors` is not supported

in `<meta>`; the `report-to` / `report-uri` browser support status; the behavior of non-ISO `Date` string parsing across engines (implementation-defined).

- **Alternative valid answers exist** for E1–E4 and C4. Grade for reasoning quality, consistency with constraints, and honest trade-offs rather than matching the model wording.
- **Suggested extension tasks** (ungraded): convert the INC-2417 postmortem action items into tickets and a ratchet script; build the Playwright legacy-API test for a real app; implement the canary gate for per-browser comparison.